COMPREHENSIVE GUIDE TO VECTORS

Programming • Mathematics • AI/ML

Created: May 27, 2026

TABLE OF CONTENTS

1. INTRODUCTION TO VECTORS

A vector is a fundamental mathematical and computational object used across multiple disciplines. In its simplest form, a vector is an ordered list of numbers (or elements) that can represent:

- **Mathematical Vector:** An element of a vector space with magnitude and direction
- **Programming Vector:** A dynamic array that can grow/shrink at runtime
- **Data Vector:** A representation of data points in n-dimensional space
- **ML Vector:** Feature representations of data for machine learning models

1.1 Why Vectors Matter

Vectors are essential because they provide a compact way to represent and manipulate data. In machine learning, vectors allow us to represent complex data (images, text, audio) in numerical form. In physics, vectors represent forces, velocities, and accelerations. In computer graphics, vectors enable transformations and 3D rendering.

2. MATHEMATICAL FOUNDATIONS

2.1 Vector Notation and Representation

Standard Notation:
• Row Vector: $v = [v1, v2, v3, ..., vn]$
• Column Vector: $v = [v1\ v2\ v3\ ...\ vn]^T$
• In mathematics: $v \in R^n$ (v is in n-dimensional real space)

Example: v = [3, 4] represents a 2D vector with components 3 and 4

2.2 Vector Operations

1. Vector Addition: $u + v = [u1+v1, u2+v2, ..., un+vn]$
2. Scalar Multiplication: $cv = [cv1, cv2, ..., cvn]$
3. Dot Product (Inner Product): $u \cdot v = u1v1 + u2v2 + ... + unvn$
4. Magnitude (Norm): $||v|| = \sqrt{(v1^2 + v2^2 + ... + vn^2)}$
5. Cross Product: (3D only) $u \times v$ gives a vector perpendicular to both
6. Vector Projection: $proj_u v = (u \cdot v / u \cdot u) \times u$

2.3 Vector Properties

• **Commutativity:** $u + v = v + u$
• **Associativity:** $(u + v) + w = u + (v + w)$
• **Distributivity:** $c(u + v) = cu + cv$
• **Zero Vector:** $0 = [0, 0, ..., 0]$ is the additive identity
• **Orthogonality:** $u \perp v$ if $u \cdot v = 0$ (perpendicular vectors)
• **Normalization:** $\hat{u} = v / ||v||$ is a unit vector in direction of v

3. VECTORS IN PROGRAMMING

3.1 Vector as Dynamic Array

In programming, a vector is implemented as a dynamic array that automatically resizes.

C++ Vector (STL):

```
#include <vector>
std::vector<int> v = {1, 2, 3};
v.push_back(4); // Add element
v.pop_back(); // Remove element
```

Python List (Dynamic Array):

```
v = [1, 2, 3]
v.append(4) # Add element
v.pop() # Remove element
```

3.2 Vector Implementation Details

Memory Allocation: Vectors allocate more capacity than needed for efficiency
Amortized O(1) Push: Adding elements is amortized constant time
O(n) Operations: Insertion/deletion in middle requires shifting elements
Cache Locality: Vectors store contiguous memory, providing good cache performance
Capacity vs Size: capacity() >= size() to avoid frequent reallocations

4. LINEAR ALGEBRA & VECTORS

4.1 Vector Spaces

Definition: A vector space V over a field F is a set with operations of addition and scalar multiplication.

Common Vector Spaces:
• R^n: n-dimensional real vectors
• C^n: n-dimensional complex vectors
• Pn: Polynomials of degree <= n
• Mmxn: m×n matrices
• C[a,b]: Continuous functions on [a,b]

4.2 Basis and Dimension

Basis: A set of linearly independent vectors that span the space
Dimension: Number of vectors in a basis
Standard Basis (R3): {(1,0,0), (0,1,0), (0,0,1)}
Linear Independence: Vectors are independent if only trivial linear combination equals zero
Span: All linear combinations of a set of vectors

5. VECTORS IN AI/ML

5.1 Feature Vectors

Definition: A feature vector is a vector of numerical features extracted from data

Example (Image Classification):
- Raw image: 224x224x3 = 150,528 pixels
- Feature vector: 1,000-dimensional vector of learned features

Example (NLP):
- Word embeddings: Each word -> 300-dimensional vector
- Sentence: Sequence of word vectors

5.2 Vector Embeddings

Common Embedding Types:
• Word2Vec: 300-dim vectors for words
• GloVe: Global vectors using co-occurrence
• FastText: Subword information
• BERT: Contextual embeddings (768 dim)
• Sentence Embeddings: 384 dimensions
• Image Embeddings: 512-2048 dimensions

6. VECTOR OPERATIONS & LIBRARIES

6.1 NumPy for Numerical Computing

Installation: pip install numpy

Basic Operations:
import numpy as np
v = np.array([1, 2, 3]) # Create vector
np.dot(v, u) # Dot product
np.linalg.norm(v) # Magnitude
v / np.linalg.norm(v) # Normalization
np.cross(v, u) # Cross product

6.2 SciPy for Advanced Linear Algebra

Installation: pip install scipy

Common Functions:
from scipy import linalg
eigenvalues, eigenvectors = linalg.eig(A)
U, S, Vt = linalg.svd(A) # Singular Value Decomposition

6.3 Machine Learning Libraries

Scikit-learn: pip install scikit-learn
from sklearn.preprocessing import normalize
from sklearn.metrics.pairwise import cosine_similarity

TensorFlow: pip install tensorflow
Provides efficient tensor/vector operations on GPU

PyTorch: pip install torch
Similar to TensorFlow, very popular for research

7. REAL-WORLD APPLICATIONS

7.1 Natural Language Processing

Word Embeddings: Each word converted to vector (king - man + woman ≈ queen)
Sentence Vectors: Average word vectors or special models
Transformer Models: BERT (768-dim), GPT (1280-dim)
Applications: Sentiment analysis, machine translation, semantic search

7.2 Computer Vision

Image Embeddings: CNN outputs (512-2048 dimensions)
Applications:
- Image retrieval: Find similar images
- Face recognition: Face embeddings for verification
- Object detection: Vector representation of objects

7.3 Recommendation Systems

User/Item Embeddings: Represent users and items as vectors
Matrix Factorization: Decompose user-item matrix
Examples: Netflix, Spotify, Amazon recommendations

8. COMPREHENSIVE STUDY ROADMAP

PHASE 1: Mathematical Foundations (2-3 weeks)

Topics: Vector basics, operations, magnitude, dot product, cross product
Resources: 3Blue1Brown YouTube, Khan Academy, Linear Algebra Done Right book
Practice: Solve 20-30 vector operation problems, use GeoGebra for visualization

PHASE 2: Linear Algebra (3-4 weeks)

Topics: Vector spaces, basis, dimension, eigenvalues, matrix decompositions
Resources: MIT OpenCourseWare (Gilbert Strang), Linear Algebra Done Right
Practice: Implement matrix operations in Python, compute eigenvalues

PHASE 3: Programming Implementation (2-3 weeks)

Topics: C++ vectors, Python lists, dynamic arrays, algorithms
Resources: C++ Primer, LeetCode (50+ array problems), GeeksforGeeks
Practice: Implement vector class, solve LeetCode array problems

PHASE 4: NumPy & Scientific Computing (2 weeks)

Topics: NumPy arrays, broadcasting, linear algebra operations
Resources: NumPy docs, DataCamp, Real Python tutorials
Practice: Convert 20 operations to NumPy, work with real datasets

PHASE 5: ML and Embeddings (4-5 weeks)

Topics: Feature vectors, embeddings (Word2Vec, BERT, etc), similarity metrics
Resources: Andrew Ng ML Specialization, Fast.ai, Hugging Face
Practice: Train word embeddings, use pre-trained BERT, build semantic search

PHASE 6: Advanced Applications (3-4 weeks)

Topics: Vector databases (Faiss, Pinecone), semantic search, recommendation systems
Projects: Build semantic search engine, recommendation system, anomaly detector
Resources: Papers with Code, Hugging Face tutorials, real-world datasets

9. KEY RESOURCES & REFERENCES

9.1 Essential Books

Mathematics:
1. "Linear Algebra Done Right" - Sheldon Axler (Best for theory)
2. "Introduction to Linear Algebra" - Gilbert Strang (Best for intuition)
3. "Linear Algebra" - Jim Hefferon (Free, comprehensive)

Programming:
4. "C++ Primer" - Lippman, Lajoie, Moo (STL vectors)
5. "Fluent Python" - Luciano Ramalho (Python sequences)

Machine Learning:
6. "Hands-On Machine Learning" - Aurélien Géron (Practical ML)
7. "Deep Learning" - Goodfellow, Bengio, Courville (Theory & practice)
8. "Speech and Language Processing" - Jurafsky, Martin (NLP free online)

9.2 Online Courses

Linear Algebra (FREE):
• 3Blue1Brown "Essence of Linear Algebra" (YouTube)
• MIT OpenCourseWare 18.06 (YouTube)
• Khan Academy: Vectors and Spaces

Programming (FREE/PAID):
• LeetCode: Arrays & Hashing Problems
• HackerRank: Data Structures
• GeeksforGeeks: Tutorials

Machine Learning (PAID with free options):
• Andrew Ng: ML Specialization on Coursera
• Fast.ai: Practical Deep Learning
• Stanford CS224N: NLP with Deep Learning (lectures free)
• Hugging Face Course on NLP (FREE)

9.3 Important Research Papers

1. "Efficient Estimation of Word Representations in Vector Space"
Mikolov et al. (2013) - Word2Vec
arXiv: 1301.3781

2. "GloVe: Global Vectors for Word Representation"
Pennington et al. (2014)

3. "Attention Is All You Need"
Vaswani et al. (2017) - Transformer

4. "BERT: Pre-training of Deep Bidirectional Transformers"
Devlin et al. (2019)

5. "Learning Transferable Visual Models From Unsupervised Text"
Radford et al. (2021) - CLIP

Access: arxiv.org, scholar.google.com, Papers with Code

9.4 Tools & Platforms

Visualization: GeoGebra, Desmos, 3Blue1Brown Manim
Practice: LeetCode, HackerRank, CodeSignal, Project Euler
ML/NLP: Hugging Face, Papers with Code, TensorFlow Hub
Vector Search: Faiss, Milvus, Weaviate, Pinecone
Documentation: NumPy, SciPy, Scikit-learn, PyTorch, TensorFlow official docs

10. QUICK REFERENCE GUIDE

10.1 Essential Formulas

Magnitude: ||v|| = sqrt(v1² + v2² + ... + vn²)
Dot Product: u·v = u1*v1 + u2*v2 + ... + un*vn
Cosine Similarity: cos(θ) = (u·v) / (||u|| * ||v||)
Euclidean Distance: d = sqrt((u1-v1)² + ... + (un-vn)²)
Normalize: û = v / ||v||
Vector Projection: proj_u(v) = ((u·v) / (u·u)) * u

10.2 Python Code Examples

NumPy:
```
import numpy as np
v = np.array([1, 2, 3])
np.dot(v, u) # Dot product
np.linalg.norm(v) # Magnitude
v / np.linalg.norm(v) # Normalize
```

Scikit-learn:
```
from sklearn.metrics.pairwise import cosine_similarity
sim = cosine_similarity([v], [u])[0][0]
```

Sentence Transformers:
```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(['text1', 'text2'])
```

10.3 C++ Vector Quick Reference

Creation:
vector<int> v; // Empty
vector<int> v(5, 10); // Size 5, value 10
vector<int> v = {1,2,3}; // Initialized

Operations:
v.push_back(x) // Add - O(1) amortized
v.pop_back() // Remove - O(1)
v[i] // Access - O(1)
v.insert(it, x) // Insert - O(n)
v.erase(it) // Delete - O(n)
v.size() // Length
sort(v.begin(), v.end()); // Sort

SUCCESS TIPS

✓ **Understand before memorizing:** Know WHY, not just HOW
✓ **Visualize:** Use GeoGebra, draw diagrams, think geometrically
✓ **Practice daily:** 30 min/day is better than 5 hours once a week
✓ **Implement from scratch:** Build vector class before using libraries
✓ **Use real data:** Apply to actual datasets from Kaggle, UCI
✓ **Join communities:** r/MachineLearning, Stack Overflow, Hugging Face
✓ **Read papers:** Start with course recommendations, not random papers
✓ **Build projects:** Complete end-to-end projects
✓ **Track progress:** Keep learning journal, review monthly
✓ **Balance theory/practice:** 40% theory, 60% implementation

ESTIMATED TOTAL TIME: 16-22 weeks (4-5 months) of consistent study
EFFORT REQUIRED: 15-20 hours per week for solid understanding